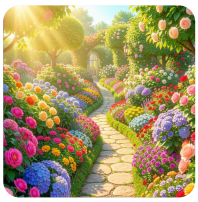


When to Water

Smart Garden Watering App — Work Sample

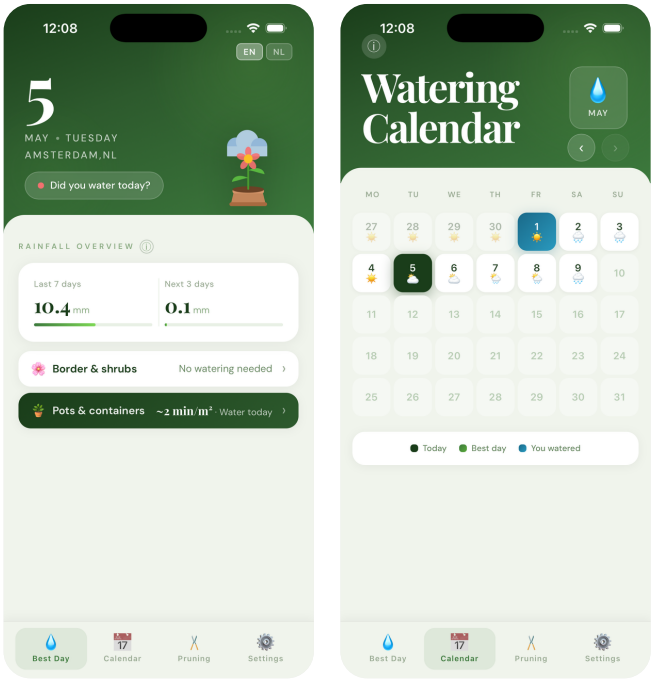
Quido Corver

Overview

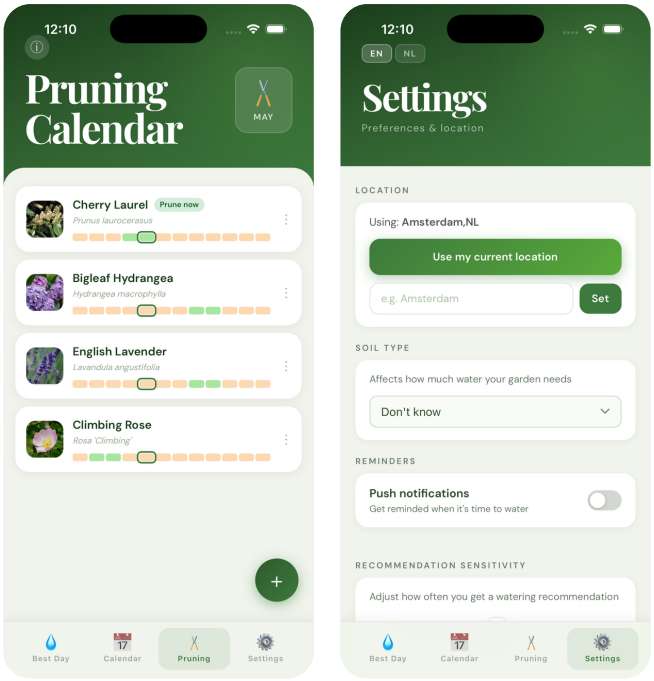
When to Water is a cross-platform mobile app (iOS & Android) that tells gardeners exactly when to water their plants — and, importantly, when not to. It combines real-time weather forecasts, 7-day rain history, plant-specific water requirements, soil type, and an evapotranspiration algorithm (Hargreaves-Samani / FAO-56) to give a personalised daily recommendation.

The app was designed, built, and shipped from concept to on-device in a matter of weeks — a self-contained MVP that demonstrates the full mobile development lifecycle: product thinking, UX design, full-stack implementation, algorithm development, and app store preparation.

App Screenshots



Left: Best Day to Water screen. Right: Calendar view with watering history.



Left: Plant library & pruning planner. Right: Settings screen.

Key Features

1. Intelligent Watering Advice

The core algorithm integrates multiple data sources to decide whether watering is needed:

- 7-day rain history via Open-Meteo (free, no API key required)
- 5-day weather forecast via OpenWeather API (proxied through a serverless Edge Function to keep the API key server-side)
- Evapotranspiration (ET_0) computed with the Hargreaves-Samani equation — the weekly water target adapts to actual temperatures and latitude, not just a calendar season factor
- Soil type adjustment (sandy / loamy / clay / chalky) increases or reduces the target by up to 50%
- Sensitivity slider lets the user fine-tune recommendations to their specific microclimate
- "Best day to water" picker finds the start of the longest upcoming dry spell, not just the first dry day — so watering happens before a rain-free stretch

2. Plant Library & Pruning Planner

Users build a personal plant library powered by two external APIs and an AI enrichment pipeline:

- Perenual plant database (250,000+ species) for search and base data
- Wikidata for Dutch common names
- iNaturalist for plant photography
- Claude AI (Anthropic) classifies plants into watering categories and extracts pruning months
- Per-plant watering multipliers (vegetables 1.75×, drought-tolerant 0.35×, potted 1.5× with reduced rain efficiency) drive the aggregate recommendation

- Monthly pruning calendar highlights which plants need attention this month

3. Push Notifications

A daily cron Edge Function (Supabase / Deno) evaluates conditions for every registered user and sends a personalised push notification via Firebase Cloud Messaging — watering advice in the morning, or a pruning reminder when plants are due.

4. Multi-Language Support

The full UI and push notification messages are available in English and Dutch, driven by a lightweight i18n module.

5. Offline Resilience

Weather data is cached in localStorage. If the network is unavailable, the app restores the last successful fetch and shows a timestamped offline banner so the user always has a usable recommendation.

Technical Stack

Frontend	React 19 + Vite (JSX, no router — tab-based navigation)
Mobile	Capacitor 8 — single codebase compiled to iOS (Xcode) and Android (Gradle)
Backend	Supabase: Postgres, Row-Level Security, anonymous auth, Edge Functions (Deno/TypeScript)
Push	Firebase Cloud Messaging (FCM) via Capacitor plugin + daily cron Edge Function
Weather data	Open-Meteo (historical, free), OpenWeather (forecast, proxied)
Plant data	Perenual API, Wikidata SPARQL, iNaturalist, Anthropic Claude Haiku (enrichment)
Testing	Vitest — 53 unit tests covering watering logic, ET_0 algorithm, best-day picker
CI/CD	Manual: npm run build → npx cap sync → Xcode / Gradle → Firebase App Distribution

---

Architecture

The app follows a local-first, API-augmented architecture. All watering logic runs on-device (shared TypeScript module imported by both the React frontend and the server-side Edge Function), making the core feature fully offline-capable.

Data Flow

- Weather forecast: React → openWeatherClient.js → weather-proxy Edge Function → OpenWeather API
- Rain history: React → openMeteoClient.js → Open-Meteo (direct, no key needed)

- Push notifications: push-daily Edge Function (cron) → fetches rain data → sends FCM
- Shared algorithm: supabase/functions/_shared/wateringLogic.ts — imported by frontend via Vite path alias and by Edge Function directly

Database (Supabase Postgres)

Seven tables, all protected by Row-Level Security policies keyed on auth.uid():

- app_users — anonymous user identities
- user_preferences — language, push enabled
- push_devices — FCM tokens per platform with cascade delete
- user_location — lat/lon for weather and ET_0 lookups
- watering_sessions — historical watering log synced from the calendar
- garden_plants — per-user plant library with watering category and pruning months
- plant_species (shared, public read) — pre-enriched species catalogue seeded from Perenual + Wikidata

---

UX & Design Approach

The app is designed around a single primary question: "Should I water today?" Every screen answers that question faster than opening a weather app and doing the mental arithmetic yourself.

- Dark-green gradient hero headers and Playfair Display serif numerals give the recommendation a premium, at-a-glance quality
- An animated SVG plant illustration reflects the current weather × soil-moisture state across six scenarios (sunny/cloudy/rain × healthy/thirsty) — making the advice emotionally legible at a glance
- iOS safe-area insets (env(safe-area-inset-*)) and font-size ≥ 16 px inputs prevent auto-zoom on focus — standard mobile UX details that matter in production
- Onboarding carousel introduces the app concept before asking for location or notification permissions
- GPS-based location detection with a manual search fallback for users who decline permissions

---

From Concept to Shipped MVP

This project illustrates the full MVP development loop that the job posting describes — starting from a tangible idea, building iteratively, and ending with a testable product on real devices.

Phased Delivery

- Phase 1 — Core algorithm & data: watering logic, weather API integration, unit tests
- Phase 2 — Frontend: React screens, calendar, offline cache, location detection
- Phase 3 — Mobile: Capacitor build, iOS + Android, push notifications via FCM
- Phase 4 — Backend: Supabase database, RLS policies, Edge Functions, cron jobs

- Phase 5 — AI enrichment: plant identification pipeline, Claude API integration, Dutch localisation
- Phase 6 — Polish: UX redesign, SVG illustrations, settings screen, app store assets

Testing

53 unit tests (Vitest) cover the watering algorithm, ET_0 computation, the best-day picker, and seasonal boundary conditions. The test suite is the safety net that makes rapid iteration safe — every algorithm change is validated before reaching the device.

Why This Is Relevant to Your Project

The job posting asks for someone who can turn a concept into a testable, iterable mobile MVP with solid UX thinking. When to Water is exactly that kind of project:

- Cross-platform mobile (iOS + Android) from a single codebase — no duplicate native work
- Real backend with authentication, a relational database, and serverless functions — not a prototype with mocked data
- External API integrations (weather, plant data, AI) managed cleanly server-side
- Thoughtful UX: the app solves a real problem simply, with accessible design and offline resilience
- Shipped to real devices and tested end-to-end, not just in a simulator

I approach every project the same way: understand the concept first, design for the user second, implement cleanly third — and communicate clearly throughout. I am happy to walk you through any part of this codebase on a call.

Quido Corver